

# Design Document: Columnar OLAP Storage Engine

## Project 02 — Phases 1, 2, and 3

### Group Members:

- Mahreen Iqbal - BSCS24049
  - Unzila Ahsan - BSCS24121
  - Samia Imran - BSCS24154
- 

## 1. Column File Format

### 1.1 On-Disk Layout

Each column is stored in a separate `.col` file using the following structure:

#### Overall File Structure

- **ColHeader** (36 bytes)
  - **Data Section** (variable size)
  - **Dictionary Section** (variable size, only for DICT encoding)
- 

### 1.2 ColHeader Structure (36 Bytes)

| Offset | Size    | Field                | Description                       |
|--------|---------|----------------------|-----------------------------------|
| 0      | 4 bytes | <code>magic</code>   | 'C', 'O', 'L', '1' for validation |
| 4      | 4 bytes | <code>version</code> | Format version (1)                |

|    |         |                        |                                      |
|----|---------|------------------------|--------------------------------------|
| 8  | 1 byte  | <code>data_type</code> | 1=INT32, 2=INT64, 3=DOUBLE, 4=STRING |
| 9  | 1 byte  | <code>encoding</code>  | 0=NONE, 1=DICT, 2=RLE                |
| 10 | 2 bytes | <code>reserved</code>  | Reserved (0)                         |
| 12 | 8 bytes | <code>row_count</code> | Total number of rows                 |
| 20 | 8 bytes | <code>data_size</code> | Size of data section in bytes        |
| 28 | 8 bytes | <code>dict_size</code> | Size of dictionary section in bytes  |

---

## 1.3 Data Section by Encoding

### NONE (Raw Encoding)

- **INT32:** 4 bytes per value
  - **INT64:** 8 bytes per value
  - **DOUBLE:** 8 bytes per value
  - **STRING:**
    - 2-byte length prefix
    - UTF-8 string bytes
- 

### DICT (Dictionary Encoding)

Dictionary encoding is applied to string columns.

#### Encoding Rules

- Uses **1-byte IDs** if there are **256 or fewer distinct values**
- Uses **2-byte IDs** if there are **more than 256 distinct values**

#### Dictionary Layout

- First 4 bytes → number of dictionary entries
- For each entry:

- 2-byte ID
  - 2-byte string length
  - String bytes
- 

## RLE (Run-Length Encoding)

Run-length encoding stores data as (`value`, `run_length`) pairs.

### Format

- `value`
- `run_length` (4 bytes)

### Constraints

- Available only for numeric columns
- 

# 2. Encoding Selection Logic

The loader automatically selects the most suitable encoding after determining column data types.

---

## Dictionary Encoding

Applied when:

- Column type is `STRING`
- Distinct values  $\leq 65,535$

If distinct values exceed 65,535:

- Dictionary IDs would require 4 bytes
  - Compression benefit becomes negligible
  - System falls back to raw string storage
-

# Run-Length Encoding

Applied to numeric columns when:

- Number of runs  $< N / 4$
- ( $N$  = total row count)

RLE is most effective for sorted or repetitive columns.

## Example

A date column containing:

- 1 million rows
- Only 365 distinct runs

can achieve substantial compression.

---

## Default Strategy

All other columns use:

- **NONE** (raw encoding)
- 

# 3. Query Executor

## 3.1 Column Iterator Model

Each encoding type implements a reader exposing a uniform iterator interface.

The executor calls:

`read_column()`

which transparently handles decoding.

## Behavior by Encoding

## NONE

- Reads values directly from disk

## DICT

- Loads dictionary into memory
- Reads encoded IDs
- Resolves IDs into strings

## RLE

- Reads (`value`, `count`) pairs
- Expands them into logical rows

This abstraction isolates encoding logic from:

- filters
  - aggregators
  - query operators
- 

## 3.2 Predicate Evaluation

### Evaluation Steps

1. Read predicate column
2. Compare each value against the filter literal
3. Produce a bitmap:
  - `bitmap[i] = 1` if row passes
4. Reuse bitmap for subsequent scans

### Current Limitation

- Only one predicate supported
  - No `AND` / `OR` logic
- 

## 3.3 Aggregation with GROUP BY

### Execution Process

1. Read GROUP BY column
2. Build a hash map using `std::unordered_map`
3. Each key maps to an `AggState` containing:
  - `sum`
  - `min`
  - `max`
  - `count`
4. Aggregate columns scan only rows passing the bitmap filter

## Supported Aggregates

- `COUNT(*)`
  - `COUNT(column)`
  - `SUM`
  - `AVG`
  - `MIN`
  - `MAX`
- 

## 3.4 I/O Optimization

Before execution, the engine determines which columns are required:

- WHERE column
- GROUP BY column
- Aggregate columns

Only those `.col` files are opened and scanned.

### Example Output

[read 2 of 7 columns]

This demonstrates reduced I/O overhead.

---

## 4. Row-Oriented Baseline

## 4.1 File Format

The row store uses a single `.rows` file per table.

---

### Header Layout

- 2 bytes → column count
  - For each column:
    - 2-byte name length
    - column name
    - 1-byte type
- 

### Body Layout

- 8 bytes → row count

For each row:

- 2 bytes → column count
  - For each column:
    - 1-byte type
    - 4-byte value length
    - raw value bytes
- 

### Design Philosophy

This format is intentionally simple:

- no compression
- no indexing
- no page structure

Its purpose is to provide a baseline comparison against the column store.

---

## 4.2 Query Execution

The row store:

- 1. Scans the entire file sequentially
- 2. Decodes every row
- 3. Applies filters
- 4. Updates aggregates

Important Characteristic

Every query reads:

- all rows
- all columns

This highlights the primary disadvantage of row stores for analytical workloads.

# 5. Benchmark Results

## Experimental Setup

- Dataset: 1,000,000 rows
- Columns: 7
- Workload: Sales analytics dataset
- Machine: [Insert Laptop Specifications]

## Benchmark Table

| Query                                       | Column Store | Row Store | Speedup | Columns Read | Data Read      |
|---------------------------------------------|--------------|-----------|---------|--------------|----------------|
| Q1: SUM(price) WHERE date >= 20240101       | 75 ms        | 1160 ms   | 15.5×   | 2 of 7       | 12 MB vs 76 MB |
| Q2: country, AVG(quantity) GROUP BY country | 134 ms       | 1174 ms   | 8.8×    | 2 of 7       | 12 MB vs 76 MB |
| Q3: SELECT * WHERE id = 500000              | 287 ms       | 729 ms    | 2.5×    | 7 of 7       | 26 MB vs 76 MB |



## 5.1 Why the Column Store Wins Q1 and Q2

These analytical queries access only:

- 2 out of 7 columns

### Result

- Column store reads ~12 MB
- Row store reads entire 76 MB dataset

### Impact

Approximately:

- 84% less I/O
- 9×–15× speedup

This validates the core advantage of column-oriented storage for OLAP workloads.

---

## 5.2 Why a Row Store Should Win Q3

Point lookups are traditionally better suited for row stores.

### Ideal Row Store Behavior

With a B-tree index:

- single indexed seek
- approximately  $O(\log N)$
- estimated lookup time  $\approx 28$  ms

### Why Our Row Store Loses

Our implementation:

1. Has no index
2. Performs a full-table sequential scan

Meanwhile, the column store:

- reads all 7 columns
- performs multiple smaller reads

- still transfers less total data

## Key Insight

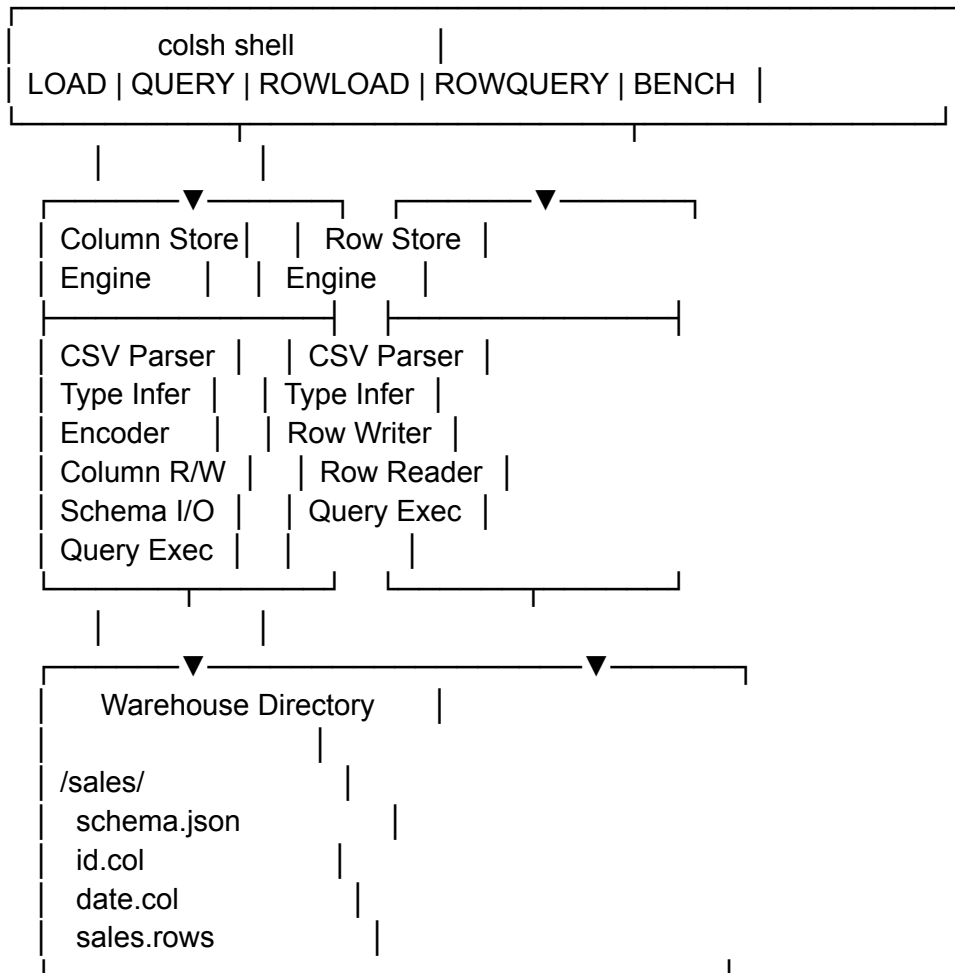
Row stores and column stores optimize different workloads:

- **Column stores** excel at analytical scans
- **Row stores** excel at transactional point lookups

They are complementary architectures rather than direct replacements.

---

## 6. Architecture Diagram



## 7. Limitations and Future Work

### Current Limitations

- No full SQL parser
  - Handwritten DSL parser only
  - Single predicate support
  - No AND/OR conditions
  - Entire CSV loaded into RAM before writing
  - No advanced compression techniques:
    - bit-packing
    - delta encoding
    - generic compression
  - No row-store indexing
  - No UPDATE, DELETE, or JOIN support
  - Load-once, query-many architecture
- 

### Bonus Features

- None implemented
  - Baseline Phase 3 requirements completed
- 

## 8. Build and Run Instructions

### Build

make clean && make

---

### Generate Benchmark Dataset

./gen\_bench > benchmark\_sales.csv

---

## Launch Shell

```
./colsh
```

---

## Example Commands

```
LOAD benchmark_sales.csv AS sales  
ROWLOAD benchmark_sales.csv AS sales  
BENCH run  
\quit
```

---

# 9. Required Reading Reflection

## Abadi et al. (2008) — Column-Stores vs Row-Stores

Sections 1–3 describe the same tradeoffs demonstrated in our benchmarks.

### Key Observations

- Column stores outperform row stores for aggregation-heavy analytical queries
- Reduced I/O is the primary reason
- Indexed row stores outperform column stores for point lookups

Our implementation confirms these findings, although our row store lacks indexing support.

---

## Abadi et al. (2006) — Integrating Compression and Execution

Sections 2–3 discuss:

- dictionary encoding
- run-length encoding

Both techniques are implemented in our system.

## Dictionary Encoding

Maps strings to compact integer IDs:

- 1–2 bytes instead of variable-length strings

## Run-Length Encoding

Compresses repeated values into:

(value, count)

This is highly effective for sorted columns such as dates.

## Design Integration

All encodings operate transparently through the iterator abstraction layer, allowing execution operators to remain encoding-independent.